OPEN AGENT HACKATHON 2026

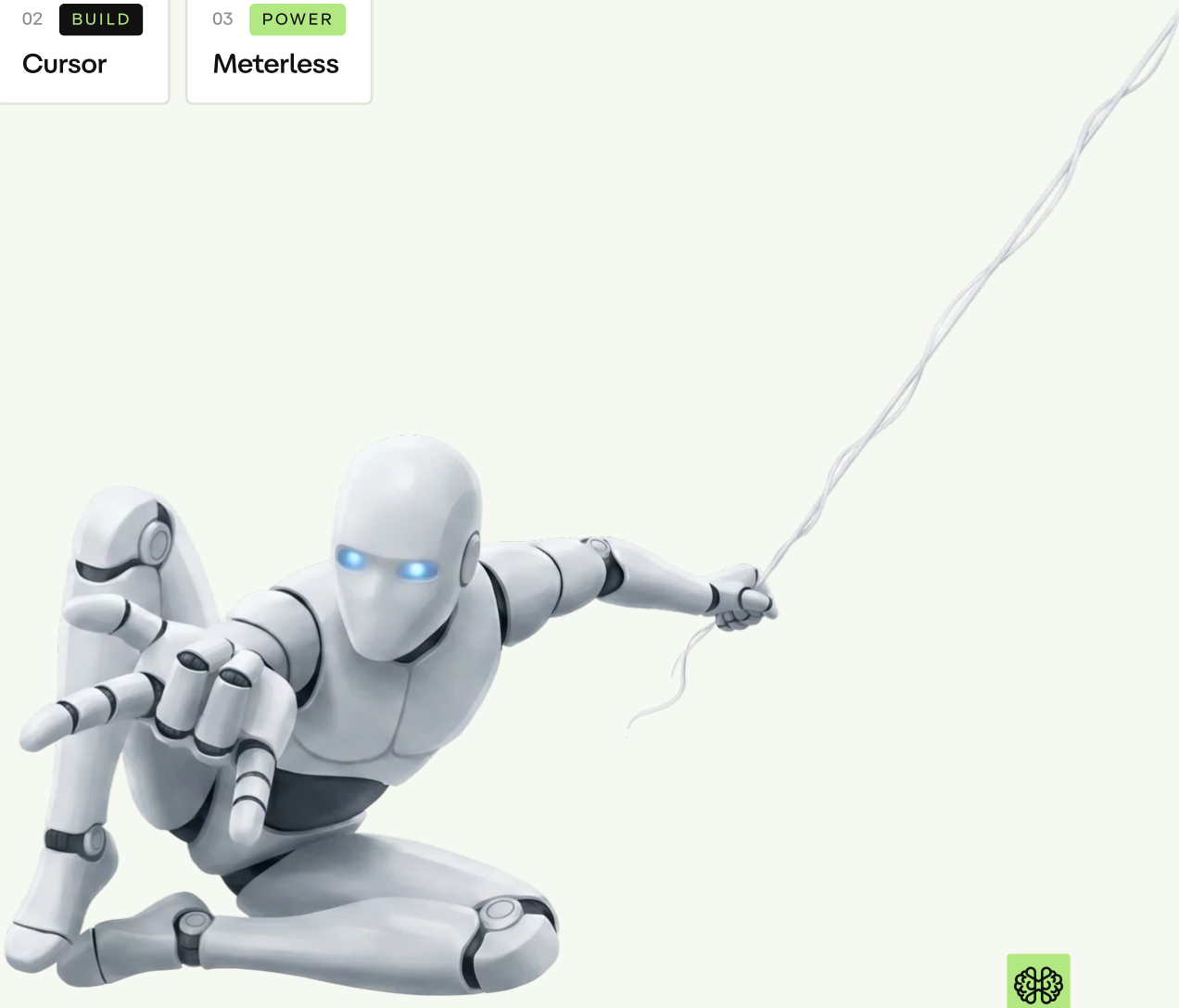
ZETARIS NVIDIA METERLESS

PARTICIPANT GUIDE · BEFORE KICKOFF

Participant Setup Guide

Set up your **Data**, **Build** and **Power** layers before the event, so the hackathon itself goes on ideas and building.

01 DATA	02 BUILD	03 POWER
Zetaris	Cursor	Meterless



Last updated

21 October 2026

GenAI.works

WELCOME

Welcome to Open Agent Hackathon 2026

Every project stands on three layers: the **Data** your agent works with, the tools you **Build** it with, and the model that **Powers** it. This guide sets up all three before kickoff, so the event itself goes on ideas and building.

Eligibility

Zetaris and Meterless must be running on your machine to be eligible for this hackathon.

By the end of this guide you will have

DATA

Zetaris running on your machine, serving the internal data your agent works with

BUILD

Cursor installed, with Grok and Composer ready on your Cursor credit, one or more Meterless Engines integrated (Markovian Thinking, H-Mem, World State), and hands-on time with the Meterless.ai prompt library

POWER

Meterless AI hotspot configured, with NVIDIA integrated as a model provider

ALL

A passing check for each layer

Contents

- The framework at a glance 03
- Before you start 04
- 01 Data: Zetaris 05
- 02 Build: Cursor and Meterless Engines 12
- 03 Power: Meterless & NVIDIA Nemotron 16
- Putting it together 21
- Verify your setup 23
- Ready for kickoff 25
- Troubleshooting and help 26
- Glossary and sources 28

Set aside an afternoon

Ideally a few days before the event. Start the Zetaris install first: it downloads large container images, and you can set up Build and Power while it runs.

Work through the guide in order. Each part ends with a **You're done when** check, so you always know whether to move on. If a term is new to you, look it up in the Glossary at the end.

THE FRAMEWORK AT A GLANCE

Use Zetaris, build in Cursor, run on Meterless

Every project stacks three layers. **Data** is what your agent knows, **Build** is where you make it, and **Power** is the model that runs it.

Required · Optional · Headline sponsor

DATA
What your agent knows

Must use

Zetaris
Internal data, runs in Docker

The data layer your agent reasons over

BUILD
Where you make it

Must use

Cursor
Grok and Composer included, paid from Cursor credit

Meterless Engines
1 or more required: Markovian Thinking, H-Mem, World State

Can also use

Meterless prompt library
Prompt files to build with

Vibe coding workshop
Learn to build, hands-on

POWER
What runs your product

Must use

Meterless
The required model provider, including the Meterless AI hotspot

Can also use

NVIDIA
Model provider via Meterless

Who pays for the AI

BUILD **While you code**
Cursor credit (Grok and Composer)

POWER **While your product runs**
Local models via Meterless Nightshift, your AI Hotspot provider

Read it left to right: data feeds the app you build in Cursor, and the finished app runs on Nemotron, reached through Meterless or NVIDIA's free API.

The one rule to remember

Cursor credit pays while you code, and your Nemotron route is used only when your product runs.

BEFORE YOU START

Do the slow item today

Register at Zetaris Cloud and download the Zetaris platform bundle and registry sign-in command from its **Run locally** page (see Data 1.1), then start the install. The first download and startup take **30 to 45 minutes**. Everything else on this list takes minutes.

LAYER	WHAT	WHY YOU NEED IT	WHERE TO GET IT
DATA	Zetaris platform bundle and registry credentials	Everything Docker needs to install Zetaris	From the Zetaris download page
DATA	Docker Desktop, version 25 or later recommended	Runs Zetaris on your machine	`docs.docker.com/get-docker`
DATA	Git	Downloads Zetaris's getting-started recipes	`git-scm.com/downloads`
DATA	WSL 2 (Windows only)	Zetaris's setup commands run in a Linux terminal	Built into Windows, see Data 1.2
BUILD	Cursor	Where you build, with Grok and Composer included	`cursor.com`
POWER	Meterless account (Option A) or a free NVIDIA account (Option B)	Your route to Nemotron, the model that runs your product	See the Power section
ALL	Python 3	Runs the smoke test and Zetaris's helper scripts	`python.org/downloads`

Hardware

Zetaris needs **8 GB of memory and 4 CPUs** given to Docker, plus at least **15 GB** of free disk space. In practice: a laptop with 16 GB of RAM or more. Apple Silicon Macs work, with a slower first start. If your laptop can't manage it, you can run Zetaris in your own AWS account instead, following Zetaris's Managed AWS cluster guide (AWS bills you for it directly): `zetaris.github.io/getting-started/docs/install/managed-aws-cluster-guide.html`

Keep your keys private

You will collect three secrets: your Zetaris password, a Zetaris API key and a Meterless API key. Cursor needs no key of your own. Store the secrets in a file called `.env` in your project folder and list `.env` in your `.gitignore`, so they never end up on GitHub. Names that start with a dot are hidden: on a Mac, press Cmd+Shift+. in Finder to show them. The Zetaris platform has its own separate `.env` inside its install folder; keep that one private too.

01 ZETARIS

Data

Zetaris for internal data

The Data rule: your project **must use Zetaris**, which serves the internal data an organisation runs on and is the center of every build. Zetaris is our headline sponsor. Start its install first.

30–45 min	8 GB · 4 CPUs	15 GB
First download and startup	Given to Docker	Free disk space

IN THIS SECTION

1.1 Get the Zetaris platform files
1.2 Prepare Docker
1.3 Unpack, sign in and run preflight
1.4 Set your administrator account
1.5 Download and start Zetaris
1.6 Verify the install
1.7 Sign in and run your first query
1.8 Load your first dataset
1.9 Connect Cursor and your app

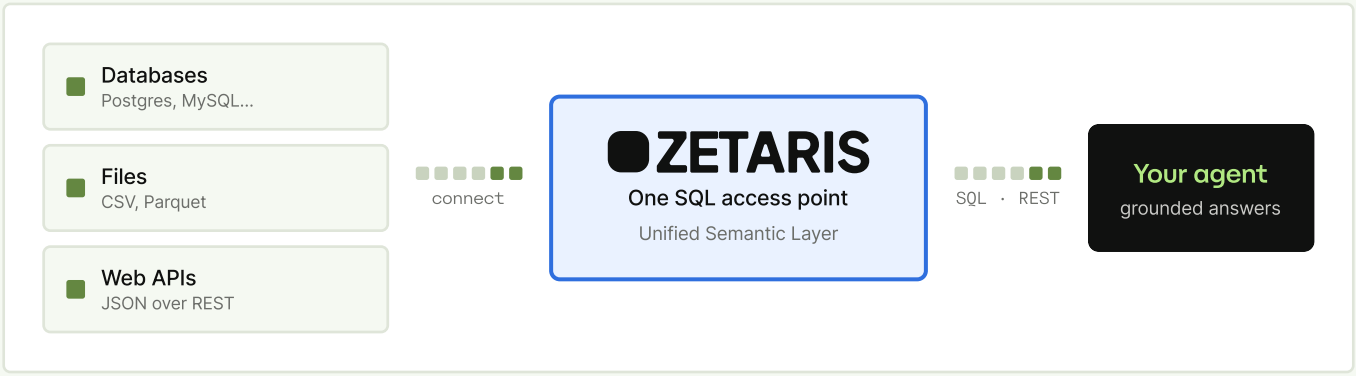
DATA HEADLINE SPONSOR

Why Zetaris belongs at the center

A model only knows what it was trained on. What an agent lacks is the data a real organisation runs on, and that is the gap Zetaris fills. These steps follow Zetaris's official getting-started repository, which also holds ready-made data recipes and a guide to connecting Cursor. Please also use Zetaris's guide to get more out of the tools: `zetaris.github.io/getting-started`

Important: hardware requirements

Zetaris needs **8 GB of memory and 4 CPUs** given to Docker, plus at least **15 GB of free disk space**. In practice that means a laptop with 16 GB of RAM or more; Apple Silicon Macs work, with a slower first start. **Less than 16 GB?** Run Zetaris as a managed AWS cluster on a new AWS Free plan account, which comes with up to US$200 in credits; in Zetaris's testing, a cluster running non-stop for about a week used roughly US$80. Follow `zetaris.github.io/getting-started/docs/install/aws-free-tier-checklist.html`. **16 GB or more?** Carry on with the guide below.



01 One access point to many sources

Zetaris connects to databases, files such as CSV and Parquet, and web APIs, then lets you query all of them with SQL from one place.

02 No pipeline per source

Your agent asks Zetaris, and Zetaris reaches the underlying systems, so you don't write and maintain a connector for each one.

03 A semantic layer for grounding

Zetaris's Unified Semantic Layer builds one consistent business view across several sources: the foundation for grounding AI answers in real data (RAG).

04 Made for this event's tracks

Two tracks, **Solving Fragmented Intelligence** and **The Agent That Can Explain Why**, ask for agents that work across scattered sources and back answers with evidence.

1.1 Get the Zetaris platform files

For this hackathon, everyone runs Zetaris locally in Docker. You need two things from Zetaris Cloud: the platform bundle, `zetaris-platform.zip`, and **registry credentials** so Docker can download the Zetaris images.

How to get them

Register at `cloud.enterprise.zetaris.com/register`, then open the **Run locally** page at `cloud.enterprise.zetaris.com/dashboard/download` to download `zetaris-platform.zip` and copy the registry sign-in command. Zetaris's full instructions: `github.com/zetaris/getting-started/blob/main/README.md`

1.2 Prepare Docker

REQUIREMENT	WHAT YOU NEED
Docker	Docker Desktop on macOS or Windows, or Docker Engine with the Compose plugin on Linux; version 25 or later recommended
Memory	8 GB given to Docker (the preflight check fails below 6 GB)
CPU	4 CPUs given to Docker
Disk	At least 15 GB free, plus room for data

What counts is how much memory **Docker Desktop is allowed to use**, set in its settings, not your laptop's total RAM.

Apple Silicon Macs

The Zetaris images are built for Intel and AMD chips, so turn on **Use Rosetta** in Docker Desktop's settings, and expect a slower first start.

Windows

Run every command below from a WSL terminal (such as Ubuntu) with Docker Desktop's WSL integration turned on. No WSL yet? Run `wsl --install` in PowerShell.

Check Docker is ready:

```terminal
docker version
docker compose version
```

1.3 Unpack, sign in to the registry and run preflight

```terminal
unzip zetaris-platform.zip
cd zetaris-platform
docker login zetarishackathon.azurecr.io
./preflight.sh
```

When `docker login` asks, enter the registry credentials from the **Run locally** page. If that page shows a different sign-in command, use it instead.

preflight.sh checks

- ✔ Docker
- ✔ Memory
- ✔ Disk
- ✔ Registry sign-in

Fix every failed check before you continue. Run all remaining Zetaris commands from this `zetaris-platform` folder.

1.4 Set your administrator account before the first start

Open the `.env` file inside `zetaris-platform` in a text editor (Cursor works well). Keep every supplied value, and set these three:

SETTING	WHAT TO PUT
`ZETARIS_ADMIN_EMAIL`	The email you'll sign in with
`ZETARIS_ADMIN_PASSWORD`	A strong password: at least eight characters, with a lowercase letter, an uppercase letter, a digit and one of `$ @ ! % * ? &` (other punctuation is rejected)
`ZETARIS_ADMIN_ORG`	Your organisation name, such as your team name

Then confirm these laptop settings, which Zetaris tested for a local install:

SETTING	VALUE
`ZSPARK_DRIVER_MEM`	`2g`
`ZSPARK_DRIVER_CORES`	2
`ZSPARK_MASTER_URL`	`local[2]`
`ZETARIS_API_MAX_HEAP`	`1g`
`UI_PORT`	3000
`ALLOWED_ORIGINS` and `GUI_URL`	`http://localhost:3000`
`COMPOSE_PROFILES`	Empty
`ZETARIS_SEED_TPCH`	`true` (keeps the bundled sample data)

> **Set these before the first start**
>
> Changing them afterwards doesn't reset an account that already exists.

Then lock the file and check it:

```
terminal
chmod 600 .env
docker compose config --quiet
```

The first command keeps `.env` private, and the second checks it's valid without printing your secrets. This `.env` belongs to the Zetaris platform: **never commit it or copy it into your project.**

1.5 Download and start Zetaris

```
terminal
docker compose pull
docker compose up -d
docker compose ps --all
```

~4 GB download	First-time setup	Total
5–15 min	5–10 min, continues after up -d	30–45 min · longer on slow Wi-Fi or Apple Silicon

When it's ready, `docker compose ps --all` shows:

```
NAME                STATUS
postgres            running
driver              running
api                 running
ui                  running
ai-datasteward      running
init                Exited (0)
bootstrap           Exited (0)
```

init and **bootstrap** are one-time setup jobs: exit code 0 means they succeeded. To watch progress, run `docker compose logs --tail=80 driver api bootstrap`.

1.6 Verify the install

```
terminal
./verify.sh
```


All **nine checks** should pass, including sign-in and a query through the web interface, the API and the query engine. If checks fail while setup is still running, wait a minute and run it again.

1.7 Sign in and run your first query

1. Open `http://localhost:3000` and sign in with the administrator email and password you set in 1.4. If you kept a supplied password, change it straight away from the user menu.
2. Open the **SQL Editor** and run `SELECT 1 AS one;` and expect one row containing 1. That confirms your login reaches the query engine.

1.8 Load your first dataset

Zetaris can turn a web API into a table you query with SQL. The starter recipe does this with **PokéAPI**, a free API that needs no key.

1. Download the getting-started repository, which holds this recipe and many more: `git clone https://github.com/zetaris/getting-started.git`
2. Open `open_data/rest_apis/sql/non_rate_limited/02_pokeapi_create.sql`.
3. In the SQL Editor, run these four statements, one complete statement at a time, waiting for each to finish:

```
SQL Editor · run one at a time
CREATE LIGHTNING DATABASE POKEAPI_REST ...
CREATE SCHEMASTORE CONTAINER pokeapi;
CREATE LIGHTNING REST TABLE pikachu_facts ...
CREATE SCHEMASTORE VIEW pikachu_abilities_table ...
```

Query the result:

```
SQL Editor
SELECT pokemon_name, ability_name, is_hidden
FROM pokeapi.pikachu_abilities_table;
```

✔ Expected: 2 rows

pokemon_name	ability_name	is_hidden
pikachu	static	false
pikachu	lightning-rod	true

The source can change, so if you see something different, look at `pikachu_facts`. Leave the rest of the file (types, stats and Charizard) for later. For more data, see the repo's recipe readiness guide. **Skip the PUDL recipe**: Zetaris lists it as known to fail.

1.9 Connect Cursor and your app to Zetaris

ROUTE	ADDRESS ON A LOCAL INSTALL	USE IT FOR
Web interface and SQL Editor	`http://localhost:3000`	Exploring data, registering sources, testing SQL
REST API	`http://localhost:8888/api/v1.0`	Your agent sending SQL and reading results as JSON
JDBC	`jdbc:zetaris:lightning@localhost:10000`	Tools that use a JDBC driver; needs Java 11 or later and the Zetaris driver file

1.9 CONNECT CURSOR AND YOUR APP · CONTINUED: REST IS THE SIMPLEST ROUTE

1. In the Zetaris web interface, create an **API key** for your user and copy it straight away (it may be shown only once). Create the key yourself; don't ask an AI agent to create or fetch it.
2. Add these settings to your project's `.env` file. In the getting-started repo, the same settings go in a file called `.env.local`.

```
.env
ZETARIS_REST_URL=http://localhost:8888/api/v1.0
ZETARIS_ORG_ID=1
ZETARIS_API_KEY=your-zetaris-api-key
ZETARIS_USERNAME=your-sign-in-email
ZETARIS_PASSWORD=your-password
```

`1` is the organisation ID on a local install. The username and password are only needed to download the API's specification.

Every REST request

POST	Authorization	X-Org-ID	X-Request-ID
/sql-editor/sqls/run	Bearer <API key>	your organisation ID	a fresh UUID

Your install serves the full API reference at `http://localhost:8888/redoc/index.html`. **Don't use port 8889**: it's internal to Zetaris.

Let Cursor do the wiring

The getting-started repo includes an `AGENTS.md` file that teaches Cursor's agent the rules for working with Zetaris, such as using Lightning SQL, sending one statement at a time and never printing credentials. Open the repo folder in Cursor, put your settings in `.env.local`, and send this as your first message:

```
Cursor · Agent chat
Read AGENTS.md and follow it. Run python3 scripts/preflight.py and show me the
result. Then connect over REST to the instance configured in .env.local and
confirm with SELECT 1 and SHOW LIGHTNING DATABASES.
Goal: <for example: onboard pokeapi>
Dry-run before creating anything and wait for my go-ahead.
```

Stop and restart Zetaris

```
terminal
docker compose stop    # stop, keeping your data
docker compose start   # resume
```

`docker compose down -v` deletes everything, including your data and encryption keys, so use it only to start over.

You're done when

`./verify.sh` passes, `SELECT 1 AS one` works in the SQL Editor, the Pikachu query returns two rows, and your Zetaris API key is saved in `.env`.

02

Build

Cursor, Meterless Engines and the workshop

The Build rule: you **must build in Cursor** and **implement at least one Meterless Engine** in a working feature of your product. Use the prompts at `meterless.ai/engines.html` to find ideas faster, then implement the engine and test that it works. Meterless prompts and engine access are also in the HackOS resources tab.

Composer	Grok	1+ Engine
Fast, everyday coding	Harder, longer-running work	Required, in a working feature

IN THIS SECTION

– The one rule: building versus running
3.1 Set up Cursor
3.2 Build with Meterless Engines
3.3 Join the vibe coding workshop
– Verify your Build setup

BUILD THE ONE RULE TO REMEMBER

Building versus running

Your project uses AI in two different ways, and each has its own setup.

BUILD

AI that helps you build

WHAT IT DOES
Writes, fixes and explains your code

MODELS
Cursor's available coding models, including **Composer** or **Grok** where offered

PAID FROM
Your **Cursor credit**; no separate provider API key needed for Cursor's included models

USED WHEN
Any time you're coding

POWER

AI that powers your product

WHAT IT DOES
Answers questions and makes decisions inside your app

MODELS
NVIDIA Nemotron and other models through **Meterless Nightshift** (see Power)

PAID FROM
Your local hardware, or the cloud provider credentials configured in **Nightshift**, subject to provider limits

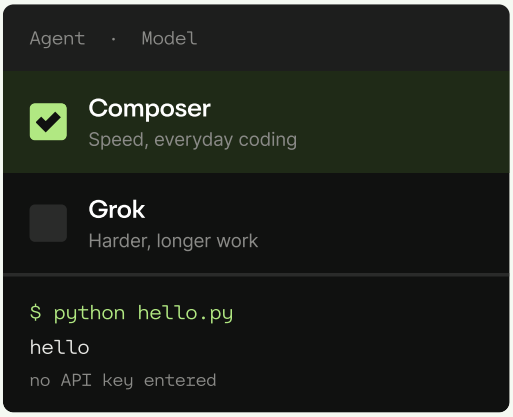
USED WHEN
When you run, power or test your product

In practice

Code with Cursor's own models, and only call the AI provider when your product actually runs. **Keep your Meterless Runtime token and NVIDIA key out of Cursor's model-provider settings.** The NVIDIA API key goes in Nightshift's Providers tab; the Nightshift Runtime URL and API key come from its Console and go into your app's Meterless provider settings (see Power).

3.1 Set up Cursor

1. Download Cursor from `cursor.com/download` (Cursor) or `cursor.com/download/bot` (Grok Bot), install it and sign in.
2. Sign up for Cursor free, and upgrade to Pro if needed.
3. Create an empty project folder, for example `open-agent-hack`, and open it in Cursor.
4. Open the Agent chat and pick a model from the model menu. Cursor recommends **Composer** for everyday coding where speed matters, and **Grok** for harder, longer-running work. Both draw on your Cursor credit, where available on your plan.
5. Ask the Agent to create and run a Python file that prints "hello". This confirms it can edit files and use your terminal.
6. New to AI coding? Work through the short **Cursor Learn** lessons on tool calling and working with agents.



3.2 Build with Meterless Engines and the prompt library

REQUIRED METERLESS

Meterless Engines are open-source building blocks that add memory, long-task reasoning, shared world state and intent routing to your applications. The Engines page gives you ready-made prompts that tell your coding agent where to find the specifications, examples and validation tools. You can combine several engines as your project grows.

METERLESS ENGINE	WHAT IT CAN ADD TO YOUR APP	EXAMPLE YOU CAN DEMONSTRATE
`H-MEM`	Persistent memory and retrieval	Your agent remembers information from an earlier session and uses it on a later task.
`Markovian`	Long-running tasks handled in bounded steps	Your app completes a multi-step job while keeping the context passed between steps controlled.
`World Model`	Shared state for agents, entities and events	Two agents read and update a common project state without losing one another's changes.
`Scout Intent`	Understanding requests, routing actions and checking risks	Your app recognizes different requests and sends them to the correct tool or workflow.

✖ **A working integration is required.** Simply mentioning an engine in your submission, copying a prompt or adding an unused dependency does not satisfy the Build rule.

Step 1 · Find a strong use case

1. Open `meterless.ai/engines.html`.
2. Have a project idea already? Start with **IDEATE → Map it onto my product**. Still deciding? Choose **IDEATE → Find my best use case**, or **The one prompt** starter at the top of the page.
3. Click **Copy**, paste the prompt into Cursor's Agent chat, and describe what you want to build.
4. Let the Agent recommend which Meterless Engine belongs in your product and how it improves the experience for your users.

Step 2 · Implement at least one engine

1. On the same page, choose **IMPLEMENT**.
2. Select the prompt that fits your project: **Give my agent memory** for H-MEM, **Long tasks, flat context** for Markovian, **Keep parallel agents in sync** for World Model, or **Route tools by intent** for Scout Intent.
3. Click **Copy** and paste the implementation prompt into Cursor's Agent chat with your project open.
4. Ask Cursor to implement the engine in your actual application, connect it to a user-facing workflow, and run the tests or conformance checks the prompt describes. Review the changes and fix any failures.
5. Starting from nothing? **IMPLEMENT → Scaffold a working agent** is another useful starting point. Keep its engine wiring intact as you adapt it.

Step 3 · Improve and verify the result

1. Use **COMBINE** prompts if you want several engines working together, such as H-MEM memory carried into a Markovian long-running task.
2. Use **HARDEN** prompts to add tests, guardrails, concurrency handling or privacy checks.
3. Run your application and show an actual feature powered by the engine: for example, an H-MEM agent that remembers a fact across sessions, or Scout Intent routing two different requests to different tools.
4. Keep a clear record of which engine you used, where it appears in your code, and the test or live demonstration showing it works.

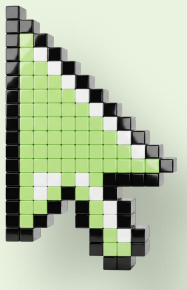
> **Important**
>
> The **POWER → Add Meterless as a provider** prompt is used in the Power section to connect your product to Nightshift. On its own, it doesn't meet the Build requirement to implement at least one Meterless Engine.

3.3 Join the vibe coding workshop

Vibe coding means building software by describing what you want to an AI agent, then reviewing and steering what it produces. Our hands-on workshop uses Cursor and the Meterless Engines prompt library to build a small agent from scratch, and it's the quickest way to learn the workflow before the clock starts.

Come with your Data, Build and Power layers set up, so you can follow along on your own machine. New to the engines? Bring one use case you want to implement, and use the workshop to get your first working feature running.

ORGANISERS 20th October, 9:00 PM PST
and joining link: https://youtu.be/Qxg5xCJIAcQ



Verify your Build setup: you're done when

- Cursor is installed and signed in, your credit is active, and the Agent has created and run a file using **Composer** or **Grok** (or another available Cursor coding model).
- You've used `meterless.ai/engines.html` to choose an engine and implementation path.
- At least one Meterless Engine is integrated into your application's working code.
- You can demonstrate the feature that uses that engine, not just describe it.
- The engine's conformance suite, evaluation or a reproducible feature test passes, and you can show the evidence.

For your submission: name the engine(s), explain what each one does in your project, and show the working feature and test result. The separate Power requirement covers connecting your app to Meterless Nightshift and, for the NVIDIA prize, using a Nemotron model through Nightshift.

Quick links: Meterless Engines and ready-made prompts (`meterless.ai/engines.html`) · Meterless open-source repository · Cursor download · Cursor Learn

03

Power

Meterless & NVIDIA Nemotron

The Power rule: your app **must support Meterless Nightshift as a primary AI provider**. Build in any language and framework, then connect its AI features to Meterless Runtime. Your app sends requests to Nightshift, and Nightshift serves them with the local or cloud models you configure.

Nightshift	NVIDIA	DGX Spark
The Meterless AI Hotspot	Added in the Providers tab	NVIDIA Nemotron Award

IN THIS SECTION

4.1 Add Meterless with the Power Prompt
4.2 Download and install Nightshift
4.3 Add NVIDIA in the Providers tab
4.4 Configure your first model lanes
4.5 Connect your app via the Console
4.6 Verify your setup
4.7 What to show judges

POWER ONLY FOR USING, POWERING AND TESTING YOUR PRODUCT

> **NVIDIA Nemotron Award: win a DGX Spark**
>
> To compete, **test powering with at least one NVIDIA Nemotron model** through Meterless Nightshift in a working feature of your app. Judging focuses on technical depth, usefulness and polish: show what Nemotron actually does inside the product, rather than simply listing it as an available model.

4.1 Add Meterless to your app with the Power Prompt

The easiest way to meet the Power rule is the ready-made **POWER → Add Meterless as a provider** prompt at `meterless.ai/engines.html`. You don't need to build the provider integration from scratch.

1. Open `meterless.ai/engines.html`, select the **POWER** category and find **Add Meterless as a provider**.
2. Click **Copy** and paste the prompt into your coding assistant, such as Cursor, Claude Code, Copilot or another agent that can edit your project.
3. Open your app's codebase in that assistant and ask it to implement the prompt, making Meterless Runtime a first-class, selectable provider and the **primary provider for your demo**.
4. Have it expose settings for **Meterless base URL**, **Meterless Runtime token** and **Meterless routing mode**. You can keep any existing providers available.
5. Have it run the prompt's connection, authorization, model-discovery and text-request checks.

The official Power Prompt specifies the OpenAI-compatible API, authentication, model discovery and error states: use the prompt from the live Engines page as your implementation contract. The NVIDIA API key is configured inside Nightshift, never hard-coded into your app.

4.2 Download and install Nightshift

What is Nightshift? It's the Meterless AI Hotspot for your computer. It serves AI requests from your apps, manages local models and connected cloud providers, and can run several model lanes for longer work. For this challenge, it's the AI backend your product calls.

1. Visit `meterless.ai` and scroll to the bottom to see the install cards.
2. Choose the Nightshift edition for your machine: Windows, Mac or DGX. **Mac M1+** is for older Macs (Big Sur, M1 to M4); **Mac M5+** is for Tahoe, M5 to M6.
3. Enter your email at the access gate, then open the confirmation email and complete verification.
4. Return to the site or follow the verified download link. Download the correct edition, install it and launch Nightshift.
5. Complete any first-run prompts, and leave Nightshift running while you test your app.

> ✖ **Use the installed Nightshift edition** with its Runtime/API hotspot. The free browser experience is useful for exploring Nightshift, but don't assume a browser demo exposes the local Runtime your app needs.

4.3 Add NVIDIA in Nightshift's Providers tab

NVIDIA provides hosted model endpoints through `build.nvidia.com/models`. Some are marked **Free Endpoint** for development and prototyping, subject to NVIDIA's access limits and terms.

1. Open `build.nvidia.com/models`, sign in and select a Nemotron model that offers an API endpoint.
2. Generate or copy your NVIDIA API key.
3. In Meterless Nightshift, open the **Setup** tab, then **Providers** in its sub-menu. The **Cloud Providers** list shows where to get each API key and where to enter it. Select **NVIDIA** from the list.
4. Paste your NVIDIA API key into the NVIDIA credential field and save it.
5. Confirm NVIDIA is connected. This unlocks NVIDIA's models in Nightshift's catalog, including the supported Nemotron models, so you can assign them to your lanes. Refresh the model list if needed.

> ✖ **The NVIDIA key goes in Providers**, not in the Console and not in your app's `.env` file. Keep it private.

4.4 Configure your first model lanes

After connecting NVIDIA, click the **NIGHT** tab at the top left of the main app menu, then open **LANES** in the left sidebar. Use each lane's **Add** button to add any model, or load a whole fleet from the **Fleets** subtab. Assign the models you want your app to use, starting with a small setup you can test.

LANE	SUGGESTED MODEL	JOB
Lane 1: Fast worker	nvidia/nemotron-3.5-lightning-30b-a3b	Quick responses, tool use and first attempts
Lane 2: Deep worker	nvidia/nemotron-3.5-super-vl-120b-a12b	More complex reasoning, planning or second opinions
Optional judge / verification lane	A suitable available Nemotron or another supported model	Check outputs against your app's requirements

These are examples, not guaranteed entitlements: select the exact Nemotron names in your NVIDIA catalog and Nightshift's model picker. You can start with just Lane 1. Local models are optional, through Nightshift's model download and lane assignment flow; you don't need a local download to use NVIDIA-hosted endpoints.

> **Make sure at least one active lane uses Nemotron**
>
> Run a short test inside Nightshift and confirm the response and the selected provider and model. For the NVIDIA award, demonstrate a task where Nemotron materially powers the result.

4.5 Open the Nightshift Console and connect your app

The **Console**, at the top right of Nightshift, is where you get the Meterless Runtime URL and API key your app needs. These are Nightshift connection credentials, separate from the NVIDIA API key you entered in Providers.

1. In Nightshift, click **Console** at the top right and find the Runtime/API connection details.
2. Copy your **Nightshift URL** (base URL) and **Nightshift API key** (Runtime access token).
3. Return to the app where you implemented Meterless as a provider with the Power Prompt.
4. Paste the Nightshift URL and API key into your app's Meterless provider settings, or into private local environment variables.
5. Select the Meterless routing mode or model exposed by Nightshift, and test the connection.

For example, if the Console shows the default local Runtime address, your app's private `.env` values might look like this:

```
.env
METERLESS_BASE_URL=http://127.0.0.1:8088/v1
METERLESS_RUNTIME_TOKEN=<your-real-nightshift-runtime-token>
METERLESS_MODEL=meterless/auto
```

- **Copy the URL shown in the Console.** `http://127.0.0.1:8088/v1` is only an example for apps on the same computer. For an app on another device, 127.0.0.1 would mean that device, so use the reachable, securely configured Nightshift URL.
- **Copy the API key shown in the Console** and use that real value as the Runtime access token, never the NVIDIA provider key or a placeholder.
- `meterless/auto` **is a routing mode, not an NVIDIA model ID.** The concrete model comes from your Nightshift provider and lane setup; use Runtime model discovery to pin an available `meterless/*` route.
- **Keep the NVIDIA API key in Nightshift's Providers tab.** Your app only needs the Meterless URL and the Nightshift API key; Nightshift handles the NVIDIA call.

> **The Runtime contract**
>
> Your coding assistant should implement the official contract from the Power Prompt: `Authorization: Bearer <token>`, the `meterless-app` header, `meterless-scope: user` for chat, `GET /v1/models` for discovery and OpenAI-compatible `POST /v1/chat/completions`. A working `/healthz` response isn't proof that a real model is connected.

4.6 Verify your setup

With Nightshift running and your token set, use your app's **Test Connection** action, or send a simple request with the actual Runtime base URL and token:

```
terminal
curl -sS http://127.0.0.1:8088/v1/chat/completions \
  -H "Authorization: Bearer $METERLESS_RUNTIME_TOKEN" \
  -H "Content-Type: application/json" \
  -H "meterless-app: hackathon-demo" \
  -H "meterless-scope: user" \
  -d '{
    "model": "meterless/auto",
    "messages": [{"role": "user", "content": "Reply with one sentence confirming this connection works."}]
  }'
```

`$METERLESS_RUNTIME_TOKEN` must be set as a shell environment variable: a `.env` file doesn't export it automatically. You can use your app's own connection test instead.

Then test the real feature end to end: your app → Meterless Runtime → active NVIDIA Nemotron lane → result shown in your app. Check Nightshift's run details, logs or receipts to confirm which model handled the request. `meterless/auto` may route to a non-NVIDIA model if several are eligible, so pin a Nemotron-backed lane or demonstrate an explicit Nemotron task to make your award evidence unambiguous.

You're done when

- Meterless is a working primary AI provider in your app, not just a dependency in your README.
- Nightshift is installed, running and reachable from the app, and your real Runtime token authenticates.
- NVIDIA credentials are saved in Nightshift's Providers tab and Nemotron models are unlocked.
- The Console supplies the Nightshift URL and API key your app uses.
- A Nemotron lane responds to a real request, and a feature in your app produces an actual answer through Meterless.
- For the NVIDIA award, the underlying run visibly uses a Nemotron model.

4.7 What to show judges

In a short demo, show **(1)** the finished app solving a useful problem, **(2)** its Meterless provider connection, and **(3)** one real request passing through Nightshift and returning a result. You can show where the app connects to the Console-provided Runtime URL, but hide all keys and tokens.

NVIDIA Nemotron Award: DGX Spark

Suggested judging criteria: technical depth, usefulness and polish. Strong projects make meaningful use of Nemotron, demonstrate a working Meterless integration and deliver a useful end-user experience. **You qualify by having Meterless integrated as an AI runtime provider in your agent or app.**

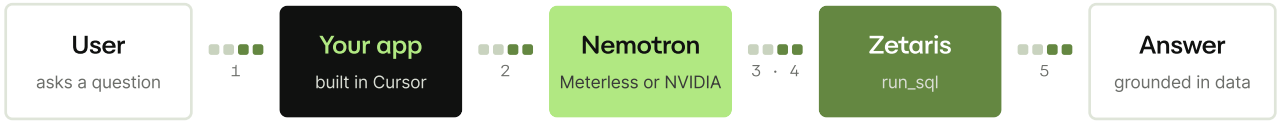
Key links: Meterless Engines and Power Prompt (`meterless.ai/engines.html`) · Nightshift downloads (`meterless.ai`) · NVIDIA model catalog and API keys (`build.nvidia.com/models`)

PUTTING IT TOGETHER

Putting Data, Build and Power together

The app you build in Cursor is a simple loop: Nemotron decides what to do, your code fetches the data it asks for from Zetaris, the result goes back to Nemotron, and the loop repeats until it can answer. Setting up this skeleton before the event means your team can start straight on ideas. Check the event rules on how much project code you may prepare in advance.

How one request moves through the layers



1		A user asks your app a question.
2	POWER	Your app sends the question to Nemotron, through Meterless or NVIDIA's free API, along with short descriptions of the tools your app offers, such as `run_sql`.
3	POWER	Nemotron replies with a tool call, for example `run_sql` with a SQL query.
4	DATA	Your code runs that call against Zetaris, and sends the result back to Nemotron.
5		Steps 3 and 4 repeat as needed, then Nemotron writes the final answer.

Build is where all of this code gets written, with Cursor's models doing the heavy lifting on your Cursor credit.

A starter project layout

project tree

```
open-agent-hack/
|-- .env              # Zetaris + model settings
|-- .gitignore        # lists .env
|-- prompts/          # optional: prompt library
|-- smoke_test.py     # the check in Verify
`-- agent/
    |-- llm.py            # Power: Nemotron
    |-- tools_zetaris.py  # Data: run_sql()
    `-- main.py           # the agent loop
```

This layout assumes **Python**, which has the widest support across all the services. JavaScript or TypeScript works just as well.

Zetaris's getting-started repo has ready-made Python and TypeScript helpers in its `scripts/` folder that you can learn from or reuse.

Scaffold it with Cursor

Give Cursor's Agent a prompt like this (attach a Meterless prompt file with `@` if you're using the library):

Cursor · Agent chat

```
Create a Python agent in the agent/ folder. In llm.py, use the openai package
with base URL MODEL_BASE_URL, key MODEL_API_KEY and model NEMOTRON_MODEL, all
read from .env. Give the agent one tool, run_sql, which sends one SQL
statement to Zetaris with a POST to ZETARIS_REST_URL + /sql-editor/sqls/run,
using ZETARIS_API_KEY as a bearer token plus X-Org-ID and a fresh UUID
X-Request-ID header, and returns the rows. Keep all secrets in .env and
explain each file.
```

Two patterns that put Zetaris at the center

Zetaris facts → Web context → Answer

01 Ground every answer in Zetaris

Your agent queries Zetaris first for the facts, and uses web data only to add context. Judges and users can trust answers that trace back to governed data, which is what **The Agent That Can Explain Why** track asks for.

Web API → Zetaris table → One SQL query

02 Bring outside data into Zetaris

Zetaris can register a web API as a table, the way the PokéAPI starter does, so a public JSON API your idea relies on can sit alongside your business data in a single SQL query.

Files on your laptop

For data you've saved as a file, put the file in storage Zetaris can reach (such as an S3 bucket) before registering it: **Zetaris can't read a file sitting on your laptop.**

You're done when

Your project folder exists, your `.env` holds all your secrets and settings, and `.gitignore` keeps them out of version control.

YOUR STACK

POWER	Meterless Nightshift · NVIDIA Nemotron
BUILD	Cursor · Meterless Engines
DATA	Zetaris · internal data in Docker

VERIFY YOUR SETUP

Three checks, two days before kickoff

Run these three checks at least **two days before kickoff**, so there's time to fix anything that fails. When all three pass, every layer is ready.

CHECK 1 DATA Zetaris is running and answers queries

From the `zetaris-platform` folder, run `./verify.sh`. Then, in the SQL Editor at `http://localhost:3000`, run `SELECT 1 AS one;` and the Pikachu query from Data 1.8.

PASS All nine `verify.sh` checks pass, `SELECT 1` returns one row, and the Pikachu query returns two rows.

CHECK 2 BUILD Cursor builds on its own models

In the Agent chat, select **Composer** or **Grok** and ask it to create and run a Python file that prints "hello".

PASS The file appears in your project and "hello" prints in the terminal, without you entering any API key.

CHECK 3 DATA POWER The all-in-one smoke test

`smoke_test.py` (next page) sends a real query to Zetaris through its REST API and a real prompt to Nemotron through your chosen route. Save it in your project folder. Your `.env` file should contain:

```
.env
ZETARIS_REST_URL=http://localhost:8888/api/v1.0
ZETARIS_ORG_ID=1
ZETARIS_API_KEY=your-zetaris-api-key
MODEL_BASE_URL=provider-address-for-your-route
MODEL_API_KEY=your-meterless-or-nvapi-key
NEMOTRON_MODEL=the-nemotron-model-id-for-your-route
```

PASS All three lines say PASS. Each FAIL line includes the error; Troubleshooting covers the common ones.

CHECK 3 DATA POWER

smoke_test.py

smoke_test.py · checks your Data and Power layers

```python
# smoke_test.py: checks your Data and Power layers
# Install once:
#   pip install openai python-dotenv requests
import os
import uuid
import requests
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
results = []

def check(name, fn):
    try:
        fn()
        results.append(("PASS", name, ""))
    except Exception as error:
        results.append(("FAIL", name,
                        str(error)[:150]))

def need(*names):
    for name in names:
        assert os.environ.get(name), \
            f"{name} is missing from .env"

def zetaris_web():
    url = os.environ.get("ZETARIS_BASE_URL",
                         "http://localhost:3000")
    requests.get(url, timeout=10).raise_for_status()

def nemotron():
    need("MODEL_BASE_URL", "MODEL_API_KEY",
         "NEMOTRON_MODEL")
    client = OpenAI(
        base_url=os.environ["MODEL_BASE_URL"],
        api_key=os.environ["MODEL_API_KEY"],
    )
    reply = client.chat.completions.create(
        model=os.environ["NEMOTRON_MODEL"],
        messages=[{"role": "user", "content":
            "Reply with the single word: ready"}],
    )
    assert reply.choices, "no reply from the model"

def zetaris_sql():
    need("ZETARIS_REST_URL", "ZETARIS_ORG_ID",
         "ZETARIS_API_KEY")
    response = requests.post(
        os.environ["ZETARIS_REST_URL"].rstrip("/")
        + "/sql-editor/sqls/run",
        headers={
            "Authorization": "Bearer "
                + os.environ["ZETARIS_API_KEY"],
            "X-Org-ID": os.environ["ZETARIS_ORG_ID"],
            "X-Request-ID": str(uuid.uuid4()),
        },
        json={"queryId": str(uuid.uuid4()),
              "sql": "SELECT 1",
              "source": "SqlEditor", "limit": 1},
        timeout=60,
    )
    response.raise_for_status()
    data = response.json().get("data")
    assert data and str(data[0][0]) == "1", \
        "unexpected reply: " + response.text[:80]

check("Data: Zetaris web interface",
      zetaris_web)
check("Data: Zetaris runs SQL over its REST API",
      zetaris_sql)
check("Power: Nemotron replies", nemotron)

for status, name, detail in results:
    print(f"{status} {name} {detail}")
```

Run it with `python smoke_test.py` (or `python3` on macOS and Linux) **while Zetaris is running.**

Each FAIL line includes the error, and the Troubleshooting section covers the common ones.

```
$ python smoke_test.py
PASS Data: Zetaris web interface
PASS Data: Zetaris runs SQL over its REST API
PASS Power: Nemotron replies
```

READY FOR KICKOFF

Tick all seven

☐	01	DATA	Zetaris Cloud account created and `zetaris-platform.zip` downloaded
☐	02	DATA	`./verify.sh` passes and the Pikachu query returns two rows
☐	03	DATA	Zetaris API key created and saved in `.env`
☐	04	BUILD	Cursor installed, credit active, and check 2 passes
☐	05	BUILD	Meterless prompt library downloaded (optional) and a workshop seat booked
☐	06	POWER	Nemotron route chosen, with its provider address, key and model ID saved in `.env`, all from the same route
☐	07	ALL	`.env` listed in `.gitignore`, and `smoke_test.py` shows three PASS lines

Where to get help

Hackathon help channel

The **Help desk** channel on HackOS: `hackathon.genai.works/messages`

Event details

The Open Agent Hackathon 2026 event page for rules, tracks and submission.

Zetaris

The getting-started repository, especially its troubleshooting and installation guides, plus the Zetaris knowledge base.

Cursor

Cursor docs.

Meterless

`meterless.ai` and the Meterless GitHub.

Nemotron

`build.nvidia.com` for NVIDIA's free API, and NVIDIA's Nemotron developer page.

When you ask for help

Include your **operating system**, the **exact command** you ran, the **full error message** and the **last step that worked**. Remove passwords, API keys and tokens before you share anything. Ask early rather than on the morning of the event.

TROUBLESHOOTING AND HELP

Find your symptom

Most setup problems fall into a handful of patterns. Find your symptom below; if it isn't listed, ask for help early rather than on the morning of the event.

LAYER	SYMPTOM	LIKELY CAUSE	FIX
DATA	**You don't have the platform bundle or registry credentials**	You haven't registered at Zetaris Cloud yet	Register at Zetaris Cloud and download them from its Run locally page. If the download isn't enabled for your account, contact Zetaris support or ask in Help Zetaris on HackOS
DATA	`./preflight.sh` reports failures	Docker isn't running, has too little memory or disk, or the registry sign-in is missing	Give Docker 8 GB and 4 CPUs, free 15 GB of disk, rerun the registry sign-in, then rerun preflight
DATA	`docker compose pull` fails with "denied" or "unauthorized"	The registry didn't accept your sign-in	Rerun `docker login` with the registry credentials from the Run locally page
DATA	`./verify.sh` fails right after starting	First-time setup is still running	Wait until bootstrap shows `Exited (0)`, wait another minute, then run it again
DATA	**Very slow first start on a Mac**	Apple Silicon runs Zetaris's Intel-built images under emulation	Turn on Use Rosetta in Docker Desktop and allow extra time
DATA	**Sign-in fails**	Wrong email or password; editing `.env` after the first start doesn't reset an existing account	Use the details you set in `.env` before the first start
DATA	**REST requests return 401**	The Zetaris API key is missing, wrong or expired	Create a new key in the Zetaris web interface and update `.env`
DATA	**REST requests return 400**	Wrong X-Org-ID, or X-Request-ID isn't a UUID	Use organisation ID 1 on a local install and a fresh UUID for every request
DATA	**"Object already exists"**	You've already run that create statement	Check the existing object matches, reuse it and move on to the next statement
DATA	**A table is created but returns no rows**	The Zetaris server can't reach the source, or the path is wrong	Check the source address, and that Zetaris itself (not just your laptop) can reach it
DATA	**A file on your laptop won't query**	Zetaris can't read files on your laptop	Put the file in storage Zetaris can reach, such as an S3 bucket, then register it

TROUBLESHOOTING · CONTINUED

LAYER	SYMPTOM	LIKELY CAUSE	FIX
BUILD	**Grok or Composer is missing from Cursor's model menu, or requests stop**	Your Cursor credit isn't active or has run out	Check your plan and usage in Cursor's settings, and ask the organisers about credit
BUILD	**You see charges on your Meterless credit while coding**	Your Meterless key ended up in Cursor's settings	Remove it from Cursor and code with Grok or Composer instead
POWER	**Nemotron calls fail even though each setting looks right**	Settings mixed from the two routes, such as a Meterless key with NVIDIA's address	Take all three settings from the same route (Power 4.5)
POWER	The smoke test says a `MODEL_` setting is missing	The setting isn't in `.env`, or `.env` isn't in the folder you ran the script from	Add the setting and run the script from your project folder
POWER	**Nemotron returns 401 or 403**	The API key doesn't match the route in `MODEL_BASE_URL`	Copy the key again from Meterless or build.nvidia.com
POWER	**Nemotron returns 404 or "model not found"**	The model ID doesn't match your route's model list	Copy the exact model ID again from Meterless or the build.nvidia.com model page
POWER	**Requests fail with a credit or rate-limit error**	Your Meterless credit has run out, or you hit NVIDIA's free-tier rate limit	Wait and retry, check your credit or limits, or switch routes by changing the three settings
ALL	`pip install` fails with "externally-managed-environment"	Your system Python blocks global installs	Create a virtual environment with `python3 -m venv .venv`, then install with `.venv/bin/pip install ...`
ALL	`python` isn't found	Python is missing or not on your PATH	Reinstall Python 3 and tick "Add to PATH" on Windows

Still stuck?

Post in the Help desk channel on HackOS with your operating system, the exact command, the full error and the last step that worked. **Remove passwords, API keys and tokens first.**

GLOSSARY

Terms used in this guide

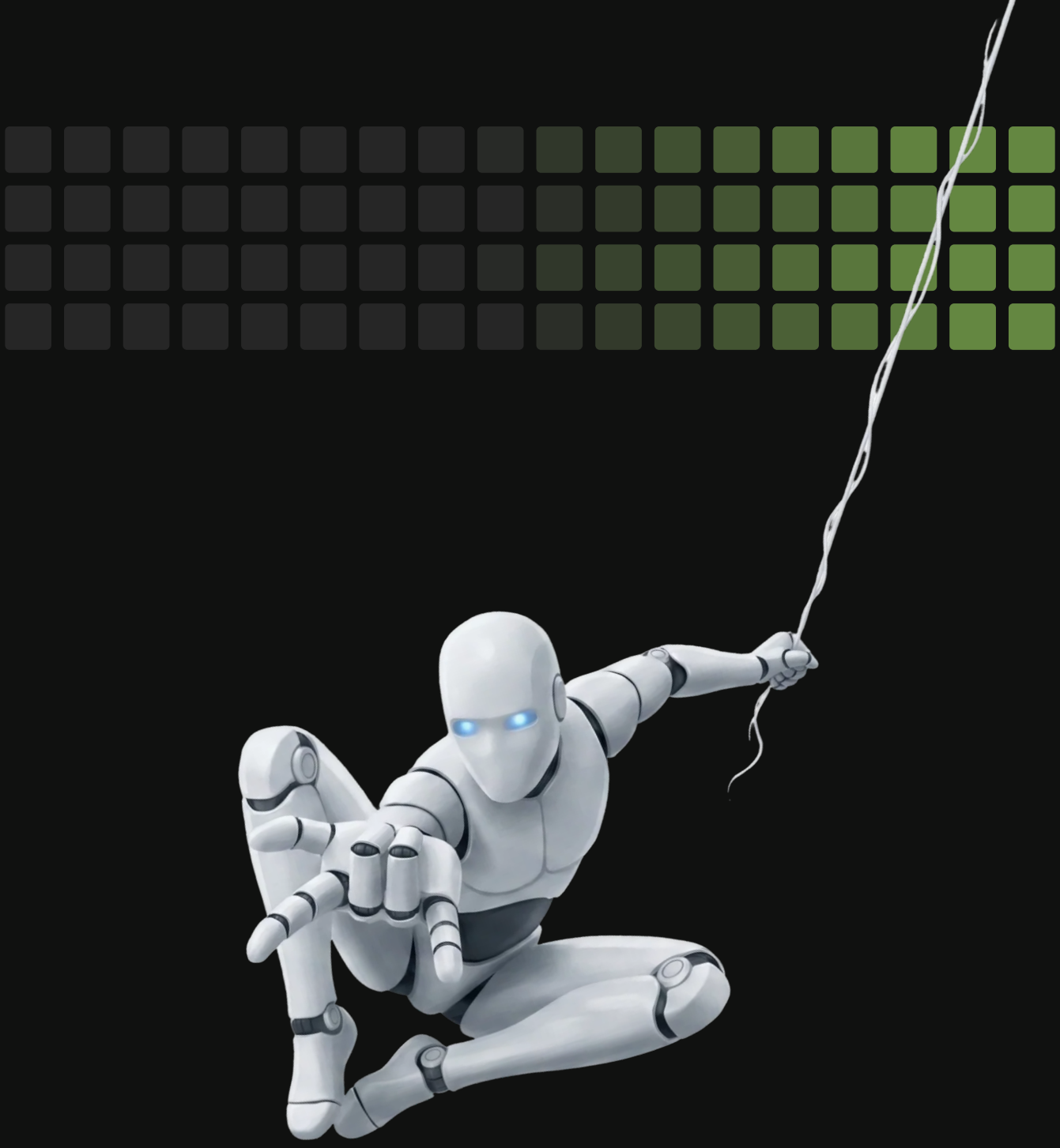
TERM	WHAT IT MEANS HERE
Agent	An app where an AI model decides which steps to take and which tools to call, then uses the results to finish a task
API	A way for one program to ask another for data or actions over the internet
API key or token	A secret password your code sends so a service knows it's you; keep it in `.env`
Base URL (provider address)	The web address your code sends model requests to; for this hackathon, Meterless's address or NVIDIA's
Composer	Cursor's own coding model, built for fast everyday coding
Container	A self-contained package that runs one piece of software, like Zetaris's query engine or web interface
Credit	Prepaid usage. Cursor credit pays for Build; Meterless credit pays for Power if you take Option A
Docker Desktop	The app that runs containers on your laptop
Docker Compose	A tool that starts several containers together from one setup file, `docker-compose.yml`
.env file	A plain text file that holds your secrets and settings so they stay out of your code
Git, clone	Git tracks code versions; cloning downloads a copy of a project, such as Zetaris's getting-started recipes
Grok	A model from xAI included in Cursor, suited to harder, longer coding tasks
IDE	Integrated development environment: the app where you write and run code, such as Cursor
JDBC	A standard way for programs to connect to databases; Zetaris accepts JDBC connections on port 10000
Lightning SQL	The SQL dialect Zetaris uses, including commands such as `CREATE LIGHTNING REST TABLE` for registering sources

GLOSSARY · CONTINUED

TERM	WHAT IT MEANS HERE
LLM	Large language model: the AI that reads and writes text, such as Nemotron
localhost and ports	localhost means your own computer; a port, such as 3000, picks which program on it you're talking to
Model provider	The service your app sends model requests to and that bills for them; here, Meterless or NVIDIA's own API
Prompt file	A saved, reusable set of instructions you hand to an AI agent, such as those in the Meterless prompt library
RAG	Retrieval augmented generation: fetching relevant data first, then giving it to the model so its answer is grounded in facts
Registry	An online store of container images; Zetaris's platform images come from a private registry you sign in to
Semantic layer	A consistent business view of data that spans several systems; Zetaris's version is the Unified Semantic Layer (USL)
SQL	The standard language for asking questions of data, such as `SELECT * FROM customers`
Terminal	The text window where you type commands: Terminal on macOS and Linux, a WSL terminal on Windows
Tool calling	A model's ability to reply with a request to run a function, such as `run_sql`, instead of plain text
UUID	A long random ID, such as `3f2a9c1e-5b7d-4e8a-9c21-7d4e5f6a8b90`; Zetaris wants a fresh one with every API request
Vibe coding	Building software by describing what you want to an AI agent, then reviewing and steering what it produces
WSL 2	Windows Subsystem for Linux: lets Windows run the Linux commands and software Zetaris needs

Sources

1	Zetaris	getting-started README, Start here, installation guide, connection options, Cursor connection guide, your first dataset, troubleshooting, project paths and event tracks, client scripts, Zetaris overview, GenAI, RAG and the semantic layer
2	Cursor	Cursor Learn, Grok in Cursor
3	Meterless	Meterless GitHub, Swarms README
4	NVIDIA Nemotron	Nemotron developer page, build.nvidia.com, n8n's NVIDIA credential guide (base URL and key steps), nemotron-ai.com (an independent site, not run by NVIDIA)



See you at kickoff.

Run `smoke_test.py` at least two days before the event. Three PASS lines and you're ready to spend the hackathon on ideas and building.

DATA Zetaris | BUILD Cursor | POWER Meterless

OPEN AGENT HACKATHON 2026

GenAI.works